

Atividade Avaliativa – AT1

Mini-Sistema de Cadastro em C

Guia didático passo a passo

Visão geral do desenvolvimento



Objetivo

Construir, em grupo, um programa em C capaz de **cadastrar, consultar, listar, alterar e excluir** informações utilizando lógica de programação, estruturas condicionais, estruturas de repetição e vetores.

Estratégia de trabalho: desenvolvam uma funcionalidade por vez. Compilem, executem e testem antes de avançar.

Como usar este guia

Este material apresenta uma sequência prática para a construção do projeto. O exemplo utilizado ao longo do guia é um cadastro de produtos, mas o grupo pode escolher outro contexto simples.

Regra de ouro: escolham de 3 a 5 informações importantes para cada cadastro. Quanto mais campos, maior será o trabalho de cadastrar, alterar, excluir e testar.

1. Escolha o tema do sistema

Definam primeiro o que o sistema irá cadastrar. O tema deve ser simples e permitir as operações de cadastro, consulta, listagem, alteração e exclusão.

Alunos	Produtos	Clientes
Livros	Veículos	Funcionários
Jogos	Filmes	Equipamentos

Pergunta para o grupo: qual informação identifica unicamente cada registro?

2. Defina os dados que serão armazenados

Depois de escolher o tema, definam os campos do cadastro e o tipo de dado adequado.

Informação	Tipo em C	Exemplo
Código	int	101
Nome	char	Mouse
Preço	float	59.90
Quantidade	int	10

3. Crie o arquivo do projeto em C

Comecem pela estrutura básica do programa:

```
#include <stdio.h>

int main() {

    // Código do sistema

    return 0;
}
```

Compilem essa primeira versão. Antes de adicionar novas partes, confirmem que o arquivo está sendo compilado e executado corretamente.

4. Crie os vetores para armazenar os dados

Cada informação do cadastro será armazenada em um vetor. No exemplo abaixo, o sistema comporta até 20 produtos:

```
int codigo[20];
char nome[20][50];
float preco[20];
int quantidade[20];

int total = 0;
```

A variável total: indica quantos registros estão realmente cadastrados. Se total = 0, não há cadastros. Se total = 3, existem três registros válidos.

5. Entenda a relação entre os vetores

Este é um dos pontos mais importantes do projeto: a mesma posição em todos os vetores representa o mesmo registro.

Vetores relacionados: a mesma posição representa o mesmo produto

Posição	0	1	2
Código	101	102	103
Nome	Mouse	Teclado	Monitor
Preço	50,00	120,00	900,00
Quantidade	10	5	3

Todos os valores da posição 1 pertencem ao mesmo cadastro

Figura 1 – Correspondência entre as posições dos vetores.

Assim, `codigo[1]`, `nome[1]`, `preco[1]` e `quantidade[1]` pertencem ao mesmo produto. Ao alterar ou excluir um cadastro, essa relação precisa ser preservada.

Atenção: não misturem posições. Se o código está na posição 2, os demais dados daquele registro também precisam estar na posição 2.

6. Crie o menu principal

O menu permite que o usuário escolha qual operação deseja realizar. Ele deve continuar aparecendo até que a opção 0 seja escolhida.

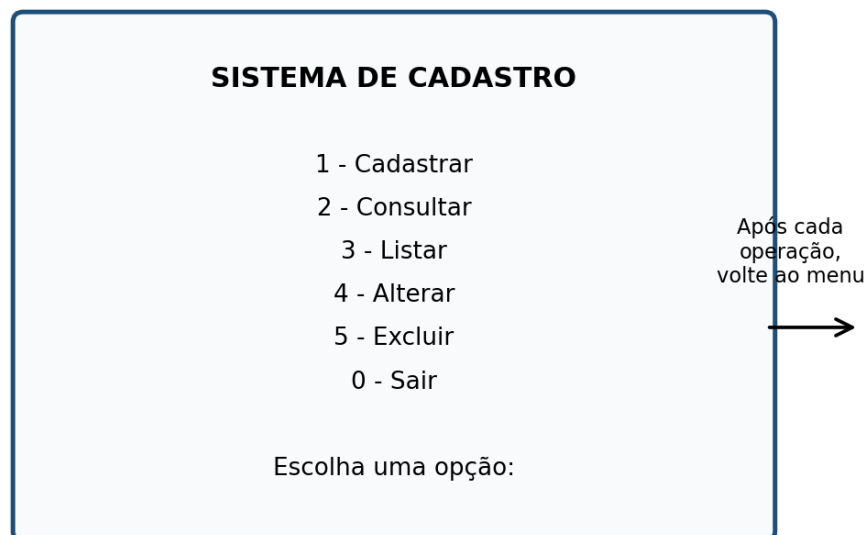


Figura 2 – Exemplo de menu principal.

```
int opcao;

do {
    printf("\n1 - Cadastrar");
    printf("\n2 - Consultar");
    printf("\n3 - Listar");
    printf("\n4 - Alterar");
    printf("\n5 - Excluir");
    printf("\n0 - Sair");

    printf("\nEscolha uma opcao: ");
    scanf("%d", &opcao);

    // operações – deve-se utilizar a estrutura switch para a execução de cada operacao
    escolhida pelo usuário.

} while (opcao != 0);
```

7. Implemente primeiro o cadastro

Comecem pela opção Cadastrar. Os dados devem ser gravados na posição indicada por total.

```
scanf("%d", &codigo[total]);
scanf("%s", nome[total]);
scanf("%f", &preco[total]);
scanf("%d", &quantidade[total]);

total++;
```

O incremento de total deve ocorrer somente depois que o cadastro for concluído. Também é necessário verificar se ainda existe espaço disponível no vetor.

Teste agora: cadastrem três registros diferentes e confirmem se total passa a valer 3.

8. Implemente a listagem

A listagem percorre somente as posições que possuem registros válidos, ou seja, de 0 até total – 1 (reflitam por que precisamos de percorrer de 0 até total -1).

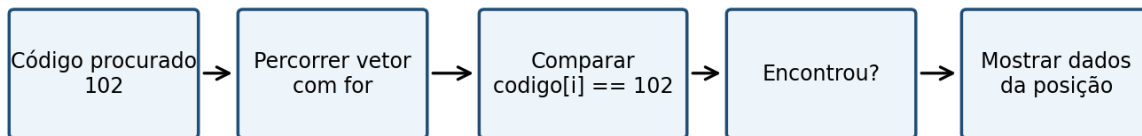
```
for (i = 0; i < total; i++) {
    printf("\nCodigo: %d", codigo[i]);
    printf("\nNome: %s", nome[i]);
    printf("\nPreco: %.2f", preco[i]);
    printf("\nQuantidade: %d\n", quantidade[i]);
}
```

Situação especial: se `total == 0`, mostrem uma mensagem informando que não existem registros cadastrados.

9. Implemente a consulta

Na consulta, o usuário informa um identificador, como o código. O programa percorre os registros e compara cada posição com o valor procurado.

Como funciona uma consulta



Se não encontrar após percorrer os registros → "Registro não encontrado"

Figura 3 – Fluxo simplificado de uma consulta.

```
for (i = 0; i < total; i++) {
    if (codigo[i] == codigoBusca) {
        // mostrar os dados da posição i
    }
}
```

Se o código for encontrado, apresentem todos os dados daquela posição. Se o laço terminar sem encontrar, informem que o registro não existe.

10. Implemente a alteração

A alteração começa exatamente como a consulta: primeiro localizem o registro. Depois, gravem os novos valores na mesma posição encontrada.

Etapa	Exemplo	O que acontece
-------	---------	----------------

1	Buscar código 102	Localizar a posição
2	Encontrado em $i = 1$	Manter a posição
3	Ler novos valores	Substituir dados em [1]
4	Exibir confirmação	Finalizar a operação

11. Implemente a exclusão

Na exclusão, não basta apagar visualmente o registro. Depois de localizar a posição, os elementos seguintes devem ser deslocados uma posição para trás.

Exclusão: os elementos seguintes ocupam a posição anterior

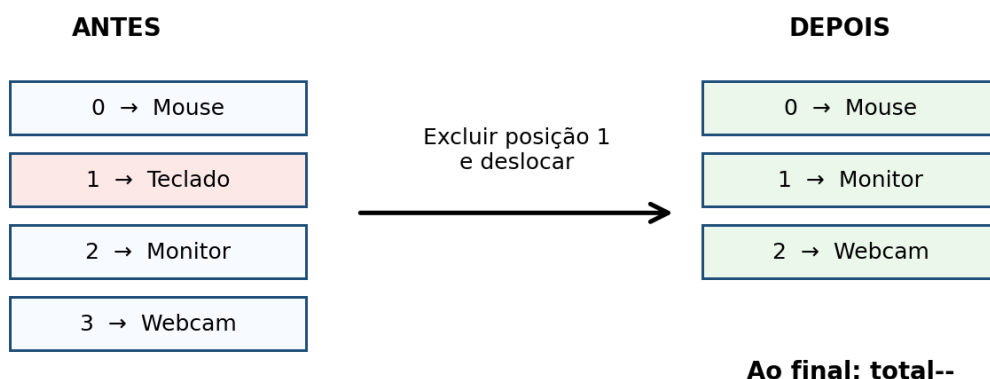


Figura 4 – Exemplo de deslocamento após a exclusão.

```
for (i = posicao; i < total - 1; i++) {
    codigo[i] = codigo[i + 1];
    preco[i] = preco[i + 1];
    quantidade[i] = quantidade[i + 1];

    // faça o mesmo com os demais dados
}

total--;
```

Muito importante: o deslocamento deve ser realizado em todos os vetores relacionados ao cadastro. Depois, total deve ser decrementado.

12. Trate situações inválidas

O sistema deve responder adequadamente a situações comuns de uso.

Situação	Resposta esperada
Opção de menu inexistente	Opção inválida.
Consulta sem resultado	Registro não encontrado.
Exclusão sem resultado	Não foi possível excluir.
Listagem vazia	Nenhum registro cadastrado.
Vetor cheio	Limite máximo de cadastros atingido.

13. Teste o projeto por etapas

Ciclo recomendado: implemente uma parte e teste antes de continuar

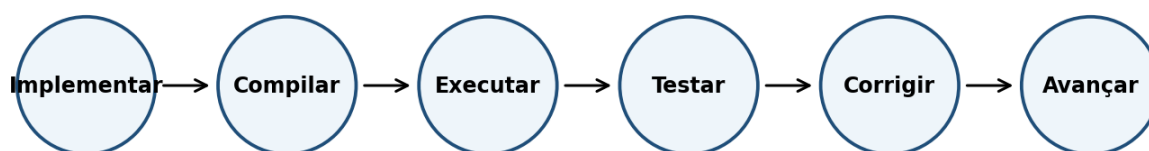


Figura 5 – Ciclo recomendado de desenvolvimento e testes.

Não deixem todos os testes para o final. A cada funcionalidade concluída, executem o programa e verifiquem se o comportamento está correto.

- Iniciar o sistema sem nenhum cadastro e tentar listar.
- Cadastrar pelo menos três registros.
- Listar e conferir os dados cadastrados.
- Consultar um registro existente e outro inexistente.
- Alterar um registro e consultar novamente.
- Excluir um registro e conferir a nova listagem.
- Tentar excluir um registro inexistente.
- Voltar ao menu após cada operação.
- Escolher 0 e confirmar que o programa encerra.

14. Organize e revise o código

Use nomes de variáveis que indiquem claramente a finalidade de cada informação e mantenha a indentação consistente.

Prefira	Evite
codigo[20]	a1[20]
preco[20]	z[20]
quantidade[20]	x1[20]
total	t
opcao	o

15. Checklist final do grupo

- ☐ Definimos um tema simples para o sistema.
- ☐ Escolhemos de 3 a 5 informações importantes.
- ☐ Criamos os vetores e a variável total.
- ☐ O menu permanece ativo até a opção Sair.
- ☐ O cadastro funciona e respeita o limite do vetor.
- ☐ A listagem mostra somente os registros válidos.
- ☐ A consulta localiza registros pelo identificador.
- ☐ A alteração modifica o registro correto.
- ☐ A exclusão desloca corretamente os elementos.
- ☐ Tratamos situações inválidas.
- ☐ Testamos todas as operações.
- ☐ O código está organizado e legível.

Resultado esperado: um programa em C que permita realizar várias operações de cadastro e retornar ao menu até que o usuário escolha Sair.

Desenvolvam o projeto com calma e por etapas. O objetivo é demonstrar a aplicação da lógica de programação, das estruturas condicionais, das estruturas de repetição e dos vetores na construção de uma solução completa.

Bom trabalho!!